

SEMI-AUTONOMOUS RED TEAMING FRAMEWORK

Backend Algorithms & Formulae Reference

The scoring, planning, policy, parsing, execution, lifecycle and cryptography mathematics of the assessment engine — stated precisely

Contents

- Contents 1
- 1 Scope, Notation & Conventions 3
 - 1.1 Scope and reading guide 3
 - 1.2 Bounding convention 3
 - 1.3 Symbol glossary 3
 - 1.4 Driver bounds and defaults 3
- 2 Finding Scoring Model 4
 - 2.1 Severity normalisation 4
 - 2.2 Driver intake 4
 - 2.3 Risk score 4
 - 2.4 Priority score 4
 - 2.5 Scoring procedure 4
 - 2.6 Worked examples 5
- 3 Deduplication & Evidence Merging 5
 - 3.1 Canonical fingerprint 5
 - 3.2 Merge operator $\otimes$ 5
 - 3.3 Analysis pipeline 5
- 4 Finding Extraction — Deterministic Per-tool Parsers 6
 - 4.1 Shared preprocessing 6
 - 4.2 nmap — host tracking and services 6
 - 4.3 nuclei — severity folding 6
 - 4.4 curl — header audit, banners, cookies 7
 - 4.5 whatweb — technology fingerprints 7
 - 4.6 sslscan — deprecated protocols 7
 - 4.7 Normalisation guards 8
- 5 Provider Analysis & Token Budgeting 8
 - 5.1 Budget allocation 8
 - 5.2 Safety and bound properties 8
- 6 Assessment Planning 8

6.1 Target classification and endpoint derivation	8
6.2 Default host plan	9
6.3 Subnet discovery and probe budget	9
6.4 Plan sources and policy filtering	10
7 Policy Engine & Scope Validation	10
7.1 Argument grammar	10
7.2 Argument scanning	10
7.3 Host normalisation	10
7.4 Scope membership	11
7.5 Resolver policy	11
7.6 Decision procedure	11
8 Engagement Letter Parsing	11
8.1 Pipeline	11
8.2 Cleaning and furniture	12
8.3 Target extraction: label table then fallback scan	12
8.4 Restriction extraction — set algebra	12
8.5 Objectives and ruled-out lists	12
9 Execution & Assessment Lifecycle	12
9.1 Streaming, caps and truncation	12
9.2 Live terminal registry	13
9.3 Timeout and process-group termination	13
9.4 Retry, staleness and uniqueness	13
9.5 Assessment status automaton	13
10 Credential Encryption & Proxy Composition	14
10.1 Fernet envelope	14
10.2 Key hierarchy	14
10.3 Proxy environment composition	14
11 Database Migration & Integrity	14
11.1 Idempotent column migration	15
11.2 Encrypt-once startup migration	15
11.3 Uniqueness repair	15
12 Tunable Constants & Bounds — Summary	15

1 Scope, Notation & Conventions

1.1 Scope and reading guide

This reference is the mathematical companion to `docs/backend_technical_reference.md`, which explains the same components in plain language. Each chapter transcribes one part of the backend: the formula or algorithm as implemented, the bounds the code enforces, and short pseudocode where the logic is procedural rather than arithmetical. All identifiers that appear below are defined in the symbol glossary (§1.3) the first time the reader needs them.

Notation is deliberately faithful to the source: a formula such as $P = \text{round}(0.40 \cdot S(\sigma) + 5 \cdot e + 0.20 \cdot \kappa + 0.15 \cdot \gamma)$ is exactly the expression evaluated in `analyzer.score_finding`, with the same coefficients. Greek lower-case letters are reserved for unitless drivers and scores; upright symbols mark constants.

1.2 Bounding convention

Every driver read from (or produced by) an external component is passed through the same bounded cast before it is used in arithmetic. Given a value v , a lower bound L , an upper bound U and a default d , the function maps

so a malformed or missing driver degrades to the declared default rather than crashing the scorer, and any out-of-range value is clamped, never allowed to pull the aggregate off its scale.

Cast semantics: `int(v)` truncates toward zero when v is a floating-point number (e.g. `int(3.9) = 3`), and raises when it is not numeric. Only the failure branch returns d ; a parseable but out-of-range integer is clamped. All bounds below are inclusive.

Throughout this document rounding means Python's `round(x)` — round-half-to-even (banker's rounding) — not the round-half-away-from-zero taught in schools. The distinction matters at ties: `round(62.5) = 62`.

1.3 Symbol glossary

e	exploitability driver, an integer in [1, 5]; how readily the issue can be weaponised (5 = trivially).
i	impact driver, an integer in [1, 5]; the harm a successful exploit does (5 = catastrophic).
x	exposure driver, an integer in [1, 5]; how reachable / internet-facing the affected surface is (5 = public).
R	raw risk score $R = e \cdot i \cdot x$, an integer in [1, 125].
σ	severity label drawn from the set {Low, Medium, High, Critical}.
$S(\sigma)$	severity base score: Low 25, Medium 50, High 75, Critical 100.
rank(σ)	severity order: Low 1, Medium 2, High 3, Critical 4.
κ	asset criticality, an integer in [0, 100]; how business-critical the affected asset appears (default 70).
γ	confidence score, an integer in [0, 100]; how sure the evidence is (default 70).
P	priority score, an integer in [15, 100]; the finding sort key used by the UI and report.
fp	deduplication fingerprint (24 hexadecimal characters, see §3.1).
N(h)	host normalisation function (§7.3): a canonical lower-case hostname or IP literal.
$\subseteq$	subnet-of relation between IPv4/IPv6 networks (<code>ipaddress.subnet_of</code>).
 	string concatenation used when building a fingerprint.
$\otimes$	merge operator that folds a duplicate finding into the already-held one (§3.2).

1.4 Driver bounds and defaults

Driver	Symbol	Bounds	Default (fallback)	Meaning when high
exploitability	e	[1, 5]	3	a public exploit or trivial trigger exists
impact	i	[1, 5]	3	confidentiality / integrity / availability loss is severe
exposure	x	[1, 5]	3	surface is reachable from the untrusted network
confidence	γ	[0, 100]	70	the evidence unambiguously supports the finding
asset criticality	κ	[0, 100]	70	asset is business-critical

severity set	The four labels and their numeric scores $S(\sigma) = \{\text{Low:25, Medium:50, High:75, Critical:100}\}$.
severity rank	$\text{rank}(\sigma)$ is the total order used to merge two reports of the same finding (§3.2).
score drivers	A finding may carry all five drivers; SCORE_DRIVERS fixes the default used if one is absent.
rounding	All aggregates round with round-half-to-even; see the worked examples in §2.6.

2 Finding Scoring Model

Source: `backend/modules/analyzer.py` (constants `SEVERITY`, `SCORE_DRIVERS`, `DEFAULT_ASSET_CRITICALITY`, and functions `bounded_int()`, `score_finding()`).

2.1 Severity normalisation

An incoming label is title-cased and accepted only if it is one of the four known levels; anything else collapses to Low. This is a validation, not a mapping: a typo such as *critical* becomes Critical after `str.title()`, while an invented label ("urgent") degrades to the least severe level.

The four levels map to a base score used in the priority aggregate and to a rank used when duplicates merge:

Level σ	Score $S(\sigma)$	rank(σ)	Intended meaning
Low	25	1	defence-in-depth weakness; no direct compromise path
Medium	50	2	exploitable under conditions; e.g. a missing security header
High	75	3	serious weakness, often directly exploitable
Critical	100	4	remote, unauthenticated compromise of a valuable asset

2.2 Driver intake

Each driver is read through the bounded cast of Eq. (1.1) with the bounds and defaults of §1.4, so arithmetic below always sees integers inside the declared ranges:

where the primed values are the raw fields reported by a parser or by the AI provider. Note that a per-target asset criticality replaces the global default 70 at intake (see §4.7): the value passed in is the target row's *criticality* column, so a business-critical host is never scored with the average 70 by accident.

2.3 Risk score

The risk score is the product of the three unitless drivers. It is the finding's quantitative footprint — how damaging the issue is (*i*) adjusted for how easy it is to trigger (*e*) and how reachable it is (*x*):

The product is stored as `risk_score`. Range: $1 \cdot 1 \cdot 1 = 1$ (benign, rarely reached) to $5 \cdot 5 \cdot 5 = 125$. The single multiplicative score keeps correlation simple — an issue that is moderately easy, moderately damaging and moderately exposed scores 27, while one that is trivial, catastrophic and internet-facing scores the full 125.

2.4 Priority score

The priority score is the weighted blend used to rank findings in the UI and the report. Written as implemented:

Because $0.25 \cdot 20 = 5$ the second term collapses to five times the exploitability driver, which is the form used below. The severity contributes at 40%, the exploitability (scaled to a 0–100 axis) at 25%, asset criticality at 20% and confidence at 15% — the four sum to 100%:

Scale bounds. The extremes are $S = 25$ (Low), $e = 1$, $\kappa = 0$, $\gamma = 0$ giving $P = 10 + 5 = 15$, and $S = 100$, $e = 5$, $\kappa = 100$, $\gamma = 100$ giving $P = 40 + 25 + 20 + 15 = 100$. So $P \in [15, 100]$. A purely Low finding can never outrank a Critical one unless its exploitability, criticality and confidence are each also at their ceilings — the four drivers pull in the same direction by design.

2.5 Scoring procedure

Algorithm 1 — score_finding(f): severity σ , risk R, priority P, confidence γ

Input : raw finding record f

Output: (σ , R, P, γ) with every driver bounded

```

1  $\sigma$    <- normalize_severity(f.severity)           // $2.1, else Low
2 S      <- SEVERITY[ $\sigma$ ]                         // 25 | 50 | 75 | 100
3 e      <- bounded_int(f.exploitability, 3, 1, 5)
4 i      <- bounded_int(f.impact, 3, 1, 5)
5 x      <- bounded_int(f.exposure, 3, 1, 5)
6  $\gamma$    <- bounded_int(f.confidence_score, 70, 0, 100)
7  $\kappa$     <- bounded_int(f.asset_criticality, 70, 0, 100)
8 R      <-  $e \cdot i \cdot x$                           // Eq. (2.4)
9 P      <- round( $0.40 \cdot S + 0.25 \cdot (20 \cdot e) + 0.20 \cdot \kappa + 0.15 \cdot \gamma$ ) // Eq. (2.5)
10 return ( $\sigma$ , R, P,  $\gamma$ )

```

2.6 Worked examples

Three findings produced by the deterministic parsers (§4) scored with the target asset criticality $\kappa = 70$, illustrating the formula and the rounding rule.

Finding (parser)	σ	$e \cdot i \cdot x$	γ	$S(\sigma)$	$R = e \cdot i \cdot x$	P (step sums)
Exposed http service, port 3000 (nmap)	Low	$2 \cdot 3 \cdot 4$	95	25	24	round($10 + 10 + 14 + 14.25$) = 48
Template-driven check matched, critical CVE (nuclei)	Critical	$4 \cdot 5 \cdot 4$	80	100	80	round($40 + 20 + 14 + 12$) = 86
Session cookie set without the Secure flag (curl)	Medium	$3 \cdot 4 \cdot 3$	90	50	36	round($20 + 15 + 14 + 13.5$) = 62 $\leftarrow$ half-to-even

The third row is the rounding tie: $0.40 \cdot 50 + 5 \cdot 3 + 0.20 \cdot 70 + 0.15 \cdot 90 = 20 + 15 + 14 + 13.5 = 62.5$, and round(62.5) = 62 under round-half-to-even. A reader summing by hand with the school rule would expect 63.

3 Deduplication & Evidence Merging

Source: backend/modules/analyzer.py — fingerprint(), _normalize(), _combine(), analyze_results().

3.1 Canonical fingerprint

Two findings describe the same defect when their title, endpoint and parameter agree after lower-casing and trimming. Those three fields are joined with vertical bars, hashed, and truncated to 24 hex characters so the key stays short enough for a database index:

m	the material string: the finding's title, endpoint and parameter, each stripped and lower-cased before hashing.
' ' separator	literal bar joins the fields, so (title = 'a b', endpoint = '') hashes differently from (title = 'a', endpoint = 'b').
truncation	SHA-256 emits 64 hex characters; only the first 24 (96 bits) are kept as the database key.

3.2 Merge operator $\otimes$

When a duplicate arrives (same fp), the held finding absorbs it by taking, per driver, the strongest value observed across the two reports. Severity follows the higher rank; the merge then lets score_finding recompute R and P so a Critical duplicate can never be filed under the severity of the copy that arrived first.

text fields	description, remediation, endpoint and parameter keep the longer of the two values; a blank never overwrites a real one.
evidence	concatenated with a newline and re-bounded to MAX_FINDING_TEXT_CHARS (20,000) so 50 reports cannot bloat one row.
source_tools	union of the two tool sets, sorted and de-duplicated.

3.3 Analysis pipeline

Algorithm 2 — analyze_results(raw): normalise, dedupe, rescore

Input : raw_outputs, optional provider credentials, asset_criticality
 Output: ranked, de-duplicated finding list (mode = deterministic-fallback | ai-provider)

```

1 if provider fully configured then
2   findings <- _ai_findings(raw)           // $5, else _fallback(raw) on any error
3   mode <- ai-provider
4 else findings <- _fallback(raw); mode <- deterministic-fallback
5
6 merged <- {}
7 for each item in findings:
8   n <- _normalize(item, asset_criticality) // $2.2 intake, truncation, fp
9   fp <- n.fingerprint
10  if fp in merged: merged[fp] <- merged[fp] * n // $3.2 strongest-wins
11  else: merged[fp] <- n
12
13 return [ score_finding(v) for v in merged.values() ] // $2.5 recompute on merged

```

4 Finding Extraction — Deterministic Per-tool Parsers

Source: backend/modules/analyzer.py. When no AI provider is configured (or the provider call fails), _fallback() dispatches each tool's captured stdout+stderr to a dedicated recogniser. Every parser emits findings that already carry the five scoring drivers, a confidence γ , evidence and remediation, so the downstream scorer needs no provider.

4.1 Shared preprocessing

Scanners colourise their output even into a pipe, and the escapes land inside the very tokens the regexes match ("SSLv3 \x1b[32menabled"). Every stream is therefore de-colourised first, and the plans also pass colour-off flags, but parsing never relies on them:

Algorithm 3 — strip_ansi(text)

```

1 return text with every CSI sequence removed
2 // pattern: \x1b [ 0-9;?]* ( [ -/*]* )? [@-~]

```

Recogniser	Trigger pattern (excerpt)	Acts per
nmap services	^port/proto open service (version)?	host block opened by "Nmap scan report for ..."
nuclei	^[template] [protocol?] [severity] url	matching line
curl headers	^Header: value inside a block opened by "HTTP/... nnn"	response block
whatweb plugins	Name[Value] tokens after "[200 OK]"	plugin token
ssllscan protocols	"SSLv2 SSLv3 TLSv1.0 TLSv1.1 ... enabled"	protocol name

4.2 nmap — host tracking and services

A subnet sweep emits one host block per live host. The parser tracks the current host from "Nmap scan report for" headers (preferring the parenthesised IP) so a per-port finding carries the host-qualified endpoint host:port/proto; otherwise every host block would collide on the same port-only endpoint and the fingerprint dedupe would silently merge all hosts but the first.

service state	open or filtered; only open counts as exposed (x = 4), filtered as limited (x = 2).
tentative match	a trailing "?" (e.g. "ppp?") is preserved in the description, strips the "?", and lowers confidence to $\gamma = 55$.
severity	Low when the service is http/https (a normal web port), else Medium.
confidence γ	95 with a version banner, 70 with a clean name, 55 when the banner match was tentative.
drivers	e = 2, i = 3 fixed; x = 4 (open) or 2 (filtered).

4.3 nuclei — severity folding

nuclei reports five levels; the framework scores four. The informational level is evidence rather than a Medium risk, so the fold maps it to Low rather than inheriting the default. Each reported level also carries a

fixed driver triple:

Reported level	Folded severity σ	Drivers (e, i, x)	Confidence γ
critical	Critical	(4, 5, 4)	80
high	High	(4, 4, 4)	80
medium	Medium	(3, 3, 4)	80
low	Low	(2, 2, 4)	80
info	Low (down-folded)	(1, 1, 3)	80

A template match is always remotely reachable — hence the steady exposure of 3–4 — and the level changes how damaging and how readily weaponised the underlying issue is. $\gamma = 80$ everywhere: signature evidence, not exploitation.

4.4 curl — header audit, banners, cookies

The header map is rebuilt at every response block, so a redirect chain is audited at the hop the client actually lands on and curl's own diagnostics (no "HTTP/... nnn" status line) never enter the map. A response that never began produces no header findings at all — a refused connection must not be reported as "all security headers missing".

Algorithm 4 — `_finding_curl(stdout)`

```

1 headers <- {}; status <- ""; in_response <- false
2 for each line in stdout:
3     if line starts with HTTP/x nnn:           // STATUS_LINE_RE
4         headers <- {}; status <- nnn; in_response <- true
5         continue
6     if in_response and line matches "Name: value":
7         headers[lower(Name)] <- value
8 if not in_response: return []                // step never reached the host
9
10 for each (name, spec) in SECURITY_HEADERS:   // §4.4b table
11     if name not in headers: emit missing-header finding(spec)
12 for name in {server, x-powered-by}:         // technology disclosure
13     if headers[name] non-empty: emit disclosure finding(name, value)
14 for each Set-Cookie line:                   // cookie flags audit
15     for flag in {httponly, secure}:
16         if flag not in lower(cookie): emit missing-flag finding(flag)

```

The audited headers, their severities and driver triples are fixed by the module-level dictionary; each missing header becomes its own finding:

Security header	Severity	(e, i, x)	Finding type
content-security-policy	Medium	(3, 4, 4)	missing-header
strict-transport-security	Medium	(3, 3, 3)	missing-header
x-frame-options	Low	(2, 3, 4)	missing-header
x-content-type-options	Low	(2, 3, 4)	missing-header
referrer-policy	Low	(2, 2, 4)	missing-header
cookie without httponly	Medium	(3, 4, 3)	missing-cookie-flag
cookie without secure	Medium	(3, 4, 3)	missing-cookie-flag
server / x-powered-by banner	Low	(2, 2, 5)	technology-disclosure

4.5 whatweb — technology fingerprints

The parser keeps only the "...Name[Value]" plugin tokens after the HTTP status block, skipping prose-like entries (country, ip, title, html5, script, email, redirectlocation). Each remaining plugin becomes a fingerprint finding at $\gamma = 85$ with $e = 2$, $i = 2$, $x = 5$ — disclosure narrows an attacker's search to the exact stack.

4.6 sslscan — deprecated protocols

The parser scans the de-colourised output for the string "Protocol enabled" for each legacy protocol. TLSv1.0/TLSv1.1 are rated Medium; SSLv2/SSLv3, which modern clients refuse entirely and which bring legacy-cipher attacks (POODLE family), are High:

Protocol	Severity	(e, i, x)	Confidence γ
SSLv2 / SSLv3	High	(3, 4, 4)	95
TLSv1.0 / TLSv1.1	Medium	(3, 4, 4)	95

4.7 Normalisation guards

count caps	at most MAX_FINDINGS = 200 findings survive any analysis; every text field is truncated at intake (title 500, prose 20,000 chars).
severity	all five parsers report through normalize_severity, so an untrusted label cannot introduce a sixth level.
AI mode	with a provider, findings from the model run through the same _normalize → merge → score pipeline; the report states which mode produced them.
per-target κ	_normalize binds a missing asset_criticality to the assessment target's value before scoring, never to the global 70.

5 Provider Analysis & Token Budgeting

Source: backend/modules/analyzer.py — `_ai_findings()`. Scanner output is concatenated into a single prompt sent to the OpenAI-compatible provider. Two caps keep the prompt bounded: one per stream, and one shared budget across the whole batch so one very chatty scanner cannot crowd every later tool out of the request.

5.1 Budget allocation

C	per-stream cap, MAX_ANALYSIS_OUTPUT_CHARS = 120,000 characters
B	remaining shared budget, MAX_ANALYSIS_TOTAL_CHARS = 400,000, initialised once per analysis

For each output stream (stdout and stderr of each tool), the retained text is the shorter of the per-stream cap and the remaining budget, and the budget shrinks by what was retained:

The two-pass form matters: stdout is consumed first, then stderr from the same tool, then the next tool. A tool whose stdout already exhausts the budget leaves nothing for its own stderr — the global cap is a hard wall, not a soft hint.

Algorithm 5 – budgeted prompt construction

Input : raw_outputs (list of {tool, stdout, stderr})
Output: prompt string embedding at most B characters of evidence

```

1 B <- MAX_ANALYSIS_TOTAL_CHARS = 400_000
2 for each output in raw_outputs:
3     stdout <- prefix(output.stdout, min(C, max(B, 0))); B <- B - |stdout|
4     stderr <- prefix(output.stderr, min(C, max(B, 0))); B <- B - |stderr|
5 prompt <- fixed instructions + JSON(bounded_outputs)
6 return prompt

```

5.2 Safety and bound properties

The provider response is treated as untrusted input, not as a model that must be obeyed: the prompt instructs it that scanner output is evidence, and the response is capped before it becomes database rows or report sections.

response shape	must parse as a JSON list of objects; any other shape falls back to the deterministic path.
count cap	at most MAX_FINDINGS = 200 findings are taken from the provider response.
text cap	title $\leq$ 500 chars; description/evidence/remediation $\leq$ MAX_FINDING_TEXT_CHARS = 20,000 each.
prompt-injection note	the system line tells the model that embedded scanner text is data, never instructions.

6 Assessment Planning

Source: backend/modules/planner.py. A plan is an ordered list of policy-legal steps. The planner either drafts steps with an AI provider and then policy-filters them, or — with no provider, on provider error, or when every model step was rejected — falls back to a fixed deterministic plan.

6.1 Target classification and endpoint derivation

The target string is first classified. A CIDR literal such as 172.28.0.0/24 is a segment, not a host, and gets its own two-step discovery plan (§6.3). Otherwise the target is parsed as a URL-ish endpoint to recover its host and optional port:

Algorithm 6 – classify and build default plan(target)

```

1  if "/" in target and ip_network(target, strict=False) parses:
2      return SUBNET_PLAN formatted with target          // §6.3, 2 steps
3
4  parsed <- urlparse(target) or urlparse("//" + target)
5  host   <- parsed.hostname or target.split(":")[0]
6  port   <- parsed.port
7  endpoint <- host if port is None else host + ":" + str(port)
8
9  for step in DEFAULT_PLAN:                          // §6.2, 7 steps
10     if step.tool in {nmap, traceroute, dig, nslookup}:
11         tgt <- host                                // name-resolving tools want the BARE host
12     else: tgt <- endpoint                            // web / TLS tools parse host:port themselves
13     step.command <- format(step.command, target=tgt, ports="-p " + port if port else "")
14     append step

```

The split matters. `nmap juice-shop:3000` and `dig +short juice-shop:3000` fail to resolve while still exiting 0, so a plan that keeps the port on those tools runs green and produces nothing. `nmap` learns the port through `-p` instead; `sslsan` and the web tools accept `host:port` and keep it.

6.2 Default host plan

Seven steps, each already policy-legal. Targets are rate-limited to 30 probes/s and colour-off flags are set so parsers see clean tokens:

#	Tool	Command template (target placeholder)	Purpose
1	nmap	<code>nmap -sV --version-light --max-rate 30 {ports}{target}</code>	exposed ports and versions
2	traceroute	<code>traceroute {target}</code>	network path / adjacency
3	dig	<code>dig +short {target}</code>	address resolution check
4	curl	<code>curl -sSI http://{target}</code>	HTTP header audit
5	whatweb	<code>whatweb -a 3 --color=never http://{target}</code>	stack fingerprinting
6	sslsan	<code>sslsan --no-colour {target}</code>	TLS protocol / cipher audit
7	nuclei	<code>nuclei -u http://{target} -tags cve,exposure,misconfig -severity medium,high,critical -rl 30 -nc -stats -duc -silent</code>	rate-limited template checks

The web-tool templates already carry `http://`, so the formatted target must not be re-prefixed — the bug that once produced `http://http://host:3000` lives in the formatting, not the tool.

6.3 Subnet discovery and probe budget

A CIDR target authorizes a range sweep, and only `nmap` accepts a range. The subnet plan is therefore two `nmap` steps — a live-host discovery then a common-port sweep with light version detection:

#	Command (target = 172.28.0.0/24)	What it establishes
1	<code>nmap -sn --max-rate 30 172.28.0.0/24</code>	which addresses are alive
2	<code>nmap -sV --version-light --open --max-rate 30 -p 22,25,53,80,110,143,443,445,3000,3306,3389,5432,5900,6379,8000,8080,8443 172.28.0.0/24</code>	exposed services on the 17 common ports

Why a /24 and not a /16? At the letter-mandated rate of $r = 30$ probes/s the probe count is the binding constraint. The discovery sweep costs about two probes per address and the service sweep one probe per (address × port):

Segment	N (addresses)	≈ probes n	≈ time t at 30/s	inside 360 s cap?
/24	256	$512 + 4,318 \approx 4,830$	≈ 161 s	yes
/16	65,536	$512 + 17 \cdot 65,534 \approx 1.1 \text{ M}$	≈ 10.5 h	no — rejected by models.py

The schema layer makes the budget a hard limit before any scan starts: a registered network target may contain at most `MAX_SUBNET_ADDRESSES = 256` addresses, which is exactly a /24 for IPv4. The executor timeout (§9.3) is the second, independent guard.

6.4 Plan sources and policy filtering

Every plan — AI or default — is reviewed against the same per-target letter restrictions in `main.create_assessment()`: steps whose tool is client-restricted for the target are dropped before the plan is stored, with the count reported back to the UI. The source tag tells the operator why a default plan was used:

Source tag	Meaning
ai-filtered	model plan survived per-step policy review
default-unconfigured	no provider key + endpoint + model set
default-provider-error	provider call or response failed
default-policy-rejected	every model step was rejected by the policy engine
user	operator supplied the plan directly

7 Policy Engine & Scope Validation

Source: `backend/modules/policy_engine.py`. The engine is an allowlist, not a blocklist: every flag a tool may receive is enumerated per tool, and anything not enumerated fails closed. Flag semantics are per tool as well, because a shared table cannot express both `nmap -A` (no value) and `curl -A` (takes a value).

7.1 Argument grammar

A tool spec declares its flag classes. When the argument scanner walks the tokens, each token is classified exactly one way:

Class	Meaning	Example
bool flag	takes no value	<code>nmap -sV</code> , <code>curl -I</code> , <code>nuclei -silent</code>
value flag	takes a value that is NOT a target	<code>nmap -p</code> , <code>nuclei -rl</code> , <code>curl -A</code>
target flag	takes a value that IS a target (scope-checked)	<code>curl --url</code> , <code>nuclei -u</code> , <code>dig -q</code>
resolver flag / sigil	takes a DNS resolver	<code>dig @8.8.8.8</code>
attached pattern	self-contained, e.g. <code>-T4</code> , <code>+short</code> , <code>-p22</code>	regex-validated token
positional	bare argument; per-tool rule decides target vs resolver	the final URL

Scanning is a single forward pass with at most one pending value. Because `argv` is a list (split by `shlex.split`), a flag and its value are two tokens and a target never needs quoting. Flag recognition also honours POSIX short-flag bundling — for `curl` only, which genuinely supports `-sSL`:

7.2 Argument scanning

Algorithm 7 — `scan_arguments(tokens, spec) → (targets, resolvers, error)`

```

1 targets <- []; resolvers <- []; positionals <- []; pending <- None
2 for each token in tokens[1:]:
3     if pending: file under pending as target/resolver; pending <- None; continue
4     if token starts with resolver sigil "@" (dig): resolvers += token[1:]; continue
5     if token is a flag:
6         match token against {bool, value, target, resolver} flag sets
7         if value/target/resolver flag: pending <- class; continue
8         if "name=value" split and name is allowed: handle; continue
9         if token matches an attached pattern (-T4, +short): continue
10        if tool supports bundling and -sSL decomposes cleanly: continue
11        else: return error(blocked flag) // fail closed
12    else: positionals += token
13 if pending: return error(pending flag starved of its value)
14 split positionals per resolver_positionals (nslookup: index ≥ 1 is a resolver)
15 return targets, resolvers, None

```

Bundle rule: a token like `-sSL` decomposes only when every character is a permitted boolean flag and, at most, the final character may take a value. A value flag in the middle fails closed rather than swallowing the rest of the token. `nuclei` and `nslookup` use single-dash long flags (`-silent`) and are never decomposed.

7.3 Host normalisation

All comparisons happen in a canonical form. A bare IPv6 literal is recognised before URL parsing (urlparse would read every colon as a port and discard the address); otherwise the host is pulled from a parsed URL, brackets and trailing dot are stripped, the name is lower-cased, and an IP literal is canonicalised so that 0:0:0:0:0:0:0:1 and ::1 compare equal:

URL vs literal	host(h) is parsed.hostname when a scheme or "://" prefix is present, else h.split(":")[0].
IP canonicalisation	if the result parses as ip_address it is returned as its canonical string (::1).

7.4 Scope membership

A target is in scope when it is inside an authorized network, equals an authorized host, or is a subdomain of an authorized host. The CIDR branch is a proper subnet test, never a base-address check:

A	the target's authorized scope set (letter scopes, else the single primary scope).
version guard	an IPv4 request is never matched against an IPv6 authorization and vice versa.
subdomain rule	h = "app.example.com" is authorized by s = "example.com"; a bare suffix on a different label boundary is rejected.

7.5 Resolver policy

A resolver argument is validated separately from targets because a DNS query aimed at a resolver is not an assessment action against it. It is accepted only when it is itself in scope or is one of the well-known public resolvers:

7.6 Decision procedure

Algorithm 8 – validate_command(command, A, expected_tool)

Input : one shell command string, authorized scopes A, optional declared tool
Output: (accepted, message, capability&risk)

```

1 reject if command empty or contains a control char (\n \r \t \x00 \x0b \x0c)
2 tokens <- shlex.split(command)           // parse error -> reject
3 reject if tokens[0] not in tool_registry  // executable not enabled
4 reject if expected_tool set and tokens[0] != expected_tool
5 (targets, resolvers, err) <- scan_arguments(tokens, rules[tokens[0]])
6 reject if err                             // blocked flag -> fail closed
7 reject if targets is empty                // no explicit target
8 reject if any target fails validate_target (Eq. 7.2 / 7.3)
9 reject if any resolver fails validate_resolver (Eq. 7.4)
10 accept; return (True, "HITL approval required", capability · risk)

```

8 Engagement Letter Parsing

Source: backend/modules/engagement_parser.py. A client letter arrives as free text once extracted from PDF/DOCX/MD/TXT. The parser recovers its structure deterministically — which hosts are in scope, their criticalities, and which tools the client ruled out per target — so what the agents "know" never depends on a provider being configured.

8.1 Pipeline

The letter is first split into clean lines (page furniture removed), then five independent extractors run over those lines. Targets are recovered first because restriction and objective extraction need to know which target a sentence names:

Algorithm 9 – parse_engagement(text)

```

1 lines <- clean_lines(text)           // drop blanks + furniture (footer, page n, "- Confidential")
2 unbulleted <- strip_bullet_glyphs_from_lines // bullet rows are still label/value rows
3 targets <- _parse_targets(unbulleted) // label table + fallback scan (§8.3)
4 targets <- _parse_restrictions(lines, targets) // allow-list / deny-list prose (§8.4)
5 fields <- _parse_doc_fields(lines, text) // engagement ref, test window
6 objectives <- _parse_objectives(lines) // "4.1 Service discovery: ..."
7 out_of_scope <- bullet_block_under("out of scope" heading)
8 prohibited <- bullet_block_under("prohibited techniques" heading)
9 return brief {client, ref, window, contact, targets, objectives, out_of_scope, prohibited}

```

8.2 Cleaning and furniture

reportlab-style page furniture repeats on every page between sections and would be read as content if left in. A line is dropped when any furniture pattern matches it:

Furniture pattern	Purpose
^~ Confidential\$	footer band on every page
^Engagement [A-Z0-9/]+\$	repeated engagement banner
^Page \d+\$	page numbers
^Request for Security Assessment Services – Confidential\$	title band

A bullet glyph that PDF extraction emits as `\x7f` (DEL) is treated as a bullet marker, so bulleted lists never parse as prose and lose their items.

8.3 Target extraction: label table then fallback scan

Targets come from two passes. The first reads RFP-style label/value rows — a label like "Authorized target address" names the next line as the address, with a CIDR tried before the host:port pattern so a subnet's mask is not left behind. The second is a fallback scan: any host:port or CIDR token anywhere in the body becomes a target unless it was already captured or sits inside the out-of-scope section:

Algorithm 10 – fallback target scan

```

1 excluded <- line indexes under the "out of scope" heading (until next numbered heading)
2 known    <- addresses already captured by the label pass
3 for each line not in excluded:
4   for each host:port match in line:      // [] host:port []
5     if match not in known: append target; known <- known u {match}
6   for each CIDR match in line:          // [] a.b.c.d/len []
7     if match not in known: append subnet target; known <- known u {match}
8 return targets

```

Set algebra keeps the two passes disjoint: known is a set growing by union, so the label pass and the fallback pass can never register the same asset twice, and the out-of-scope set is subtracted up front so "out of scope: the corporate network 10.10.0.0/16" never becomes a scan target.

8.4 Restriction extraction — set algebra

Per-target tool restrictions are read from the rules-of-engagement prose, which takes two sentence shapes. Known tools are the eight the framework can run: nmap traceroute dig nslookup curl whatweb sslscan nuclei. For every target a section names, a sentence contributes a restriction only when it also names at least one known tool:

K	the eight known tools; any tool named outside K is already refused by the policy engine.
K_S	the known tools named inside the sentence, detected by whole-word match.
R_t	the restriction set per target — later enforced at plan time and again at execute time.
complement	an allow-list sentence bans every known tool EXCEPT those it names.

8.5 Objectives and ruled-out lists

Numbered objective items of the form "4.1 Service discovery: ..." are captured by a heading regex plus a colon filter — the colon is what separates an objective from a same-shaped section heading such as "3.1 Primary asset — ...". A wrapped continuation line is folded back into the previous item when that item does not yet end a sentence. Out-of-scope and prohibited-technique lists are collected from bullet blocks under their headings, and items start only on bullet lines so a section's intro sentence is never mistaken for an item.

9 Execution & Assessment Lifecycle

Source: backend/modules/executor.py and backend/main.py. Commands run as real subprocesses (never through a shell), streamed into bounded buffers and killed on timeout.

9.1 Streaming, caps and truncation

Algorithm 11 — execute_command(tool, command, env, execution_id)

```

1 start timer; chunks <- []
2 process <- create_subprocess_exec(*shlex.split(command), pipes, env)
3 spawn two drain coroutines (stdout, stderr) + process.wait() in parallel
4 wait for all with timeout = EXECUTION_TIMEOUT_SECONDS = 360 s
5 on timeout: kill process group (SIGKILL on POSIX); return_code <- -1; timed_out <- true
6 stdout <- truncate(chunks.stdout); stderr <- truncate(chunks.stderr)
7 if timed_out: prefix stderr with the timeout notice
8 return {tool, command, stdout, stderr, return_code, duration_ms}

```

Draining reads 64 KiB chunks but retains at most `MAX_OUTPUT_BYTES = 800,000` bytes per stream (200,000 chars × 4, an estimate of the worst-case UTF-8 width), continuing to read so a chatty scanner never blocks on a full pipe. The decoded text is finally truncated to `MAX_OUTPUT_CHARS = 200,000` characters with a marker appended:

9.2 Live terminal registry

While a command is in flight, its partial output is kept in memory so the UI can stream it. The live buffer is a sliding window — appending and keeping only the most recent characters:

The registry holds only genuinely running commands: entries are registered before the process starts and removed however it ends, so a later poll that finds no entry knows the run is over and re-reads the authoritative database row.

9.3 Timeout and process-group termination

Both nmap and nuclei spawn children, so a timeout kills the whole process group, not just the parent. The 360 s figure is measured against the nuclei step — 12,078 requests across 5,242 templates is about 308 s at 30 req/s — so the cap leaves headroom without letting a hung scanner wait meaningfully longer. On a client disconnect the coroutine is cancelled; the finally-branch terminates any process still running so a scan can never continue hammering the target with nobody left to read it.

Execution aspect	Value / rule
timeout	360 s (EXECUTION_TIMEOUT_SECONDS); on expiry return_code = -1 and a notice is prefixed to stderr
output cap	200,000 chars per stream (stdout, stderr) after drain
read chunk	64 KiB (READ_CHUNK_BYTES)
live buffer	last 200,000 chars (LIVE_BUFFER_CHARS), in-memory only
env sandbox	subprocess inherits only PATH/HOME/... + proxy vars, never DATABASE_URL or provider keys

9.4 Retry, staleness and uniqueness

Each plan step has at most one execution row per assessment (UniqueConstraint on assessment_id + step_index), which doubles as a concurrency guard. A step may be re-approved only when its previous attempt failed or was abandoned. An execution row whose command can no longer be alive is stale and reclaimable:

Retryable ⇔ return_code ≠ 0 and (the row is complete, or it is stale). A successful step is never retried, keeping the audit trail append-only in the case that matters. Re-running a step clears its old output and invalidates any analysis derived from it.

9.5 Assessment status automaton

An assessment moves through a small set of states. Let E = enabled step indexes and X = indexes with a completed execution. Analysis is gated on $X \supseteq E$, and the plan is frozen once any execution exists:

Transition	Guard
awaiting_approval → running	a step is approved and dispatched (per-step)
running → awaiting_approval	enabled steps remain unexecuted after a step returns
running → ready_for_analysis	$X \supseteq E$: every enabled step has a completed execution
→ analyzed	analysis endpoint runs; findings written; analysis_mode recorded

Transition	Guard
analyzed → reported	report rendered to data/reports/report_<id>.html
any pre-run state → awaiting_approval	plan edited (rejected once $X \neq \emptyset$)
stale running → (recovered)	reconcile_running_status moves it off "running"

Two gates look like bugs during a demo but are deliberate: the plan is frozen once any step has executed (plan edits return 409), and analysis requires every enabled step to have run. Disable steps before starting for a shorter run.

10 Credential Encryption & Proxy Composition

Source: `backend/modules/secret_store.py`. The provider API key and proxy password are stored encrypted at rest, because the SQLite file is ordinary data: it gets copied into backups, mounted into containers, and occasionally attached to a bug report.

10.1 Fernet envelope

Values use the Fernet symmetric-key format (AES-128-CBC with PKCS#7 padding and HMAC-SHA256 authentication). A Fernet token is one base64url blob whose binary content is laid out as:

```
token = b64url( 0x80 || timestamp t (8 bytes, big-endian seconds) || iv (16 bytes) || ciphertext AES-128-CBCk(m) || HMAC-SHA256k(0x80 || t || iv || ciphertext) )
```

The framework stores the token with a versioning prefix so plaintext credentials written before encryption was introduced are recognised rather than fed to Fernet (which would reject them and silently discard a key the operator had already configured):

10.2 Key hierarchy

Key source	When used	Notes
REDTEAM_SECRET_KEY (env)	always, when set	never lands on the same volume as the database; rotated key file takes effect without restart
data/.secret_key (file)	otherwise, generated on first use	Fernet.generate_key(); chmod 0600 (Linux); read again on every use — deliberately uncached

A malformed master key raises rather than degrading to plaintext — storing a secret unencrypted because the key was mistyped is the one outcome worse than refusing the write. A decrypt that fails (rotated or lost key) degrades to "no provider configured":

Algorithm 12 – secret store

```
encrypt(v):
1  if v is empty: return ""
2  if is_encrypted(v): return v           // never double-encrypt
3  return "enc:v1:" + Fernet(load_key()).encrypt(v)

decrypt(v):
4  if v is empty: return ""               // nothing stored
5  if not is_encrypted(v): return v        // legacy plaintext passes through
6  try: return Fernet(load_key()).decrypt(v[7:]) // v[7:] drops "enc:v1:"
7  except (InvalidToken, ValueError): return "" // key lost → provider looks unset
```

10.3 Proxy environment composition

When a proxy is configured with a username and password, the two secrets are percent-encoded into the authority component of the URL and the result is exported to the scanner subprocess through all four spellings of the proxy variables:

with q the RFC 3986 percent-encoding of each segment, so a password containing `:` or `@` cannot corrupt the URL authority.

11 Database Migration & Integrity

Source: `backend/database.py`. SQLAlchemy `create_all` only creates missing tables, so columns added after the first release are applied by an idempotent ALTER pass at import time.

11.1 Idempotent column migration

For each table, the columns already present are read once and only missing columns are added — running the migration a second time is a no-op because the "existing" set already contains every column:

11.2 Encrypt-once startup migration

On startup any credential still stored as plaintext (written before encryption existed) is encrypted in place; legacy baked-in endpoint/model defaults are cleared only when no key was ever saved, so the operator is asked for their own provider rather than silently inheriting a third-party endpoint nobody chose.

11.3 Uniqueness repair

The execution uniqueness constraint is created only when no duplicates exist, detected with a grouped count — if any (assessment_id, step_index) group has more than one row the constraint is left for manual reconciliation instead of failing the migration:

12 Tunable Constants & Bounds — Summary

Constant	Value	Module	What it bounds
MAX_PLAN_STEPS	50	planner / main	maximum steps in a plan
MAX_PLAN_FIELD_CHARS	100 / 4000 / 2000	main	tool / command / reason length per step
MAX_ANALYSIS_OUTPUT_CHARS	120,000	analyzer	per-stream chars in an AI prompt
MAX_ANALYSIS_TOTAL_CHARS	400,000	analyzer	shared AI prompt budget
MAX_FINDINGS	200	analyzer	findings kept after analysis
MAX_FINDING_TEXT_CHARS	20,000	analyzer	prose field length and merge cap
DEFAULT_ASSET_CRITICALITY	70	analyzer	κ when the target declares none
EXECUTION_TIMEOUT_SECONDS	360	executor	per-command run cap
MAX_OUTPUT_CHARS / _BYTES	200,000 / 800,000	executor	persisted per-stream cap
LIVE_BUFFER_CHARS	200,000	executor	in-memory live-terminal window
READ_CHUNK_BYTES	65,536	executor / main	stream and upload chunk size
EXECUTION_STALE_AFTER	420 s	main	staleness = timeout + 60 s
MAX_SUBNET_ADDRESSES	256	models	largest CIDR target (a /24)
MAX_AUTHORIZED_SCOPES / _CHARS	50 / 255	models	scope list size and width
MAX_RESTRICTED_TOOLS / _CHARS	20 / 64	models	letter tool restrictions
MAX_REQUIREMENT_BYTES	5 MiB	main	uploaded file size
MAX_BRIEF_ITEMS / _CHARS	100 / 20,000	models	engagement brief bounds
objective / requirements	1000 / 30,000 chars	models	prompt field width
SEVERITY / SCORE	25 · 50 · 75 · 100	analyzer	σ base scores
SECRET_PREFIX	enc:v1:	secret_store	encrypted-value marker

These are the constants as shipped; every one is tunable at the module top of the file named in the third column. Two interact: the /24 sweep budget (\$6.3) and the 360 s executor cap are sized together so a full default-plan run on a /24 finishes inside the window.